

LexiSense: Comprehensive Technical & Security Review

Exhaustive Technical Breakdown of Technology Stack, Multimodal 3-Pillar Triangulation Algorithm, and Security/Privacy Posture

Target System: LexiSense Early Dyslexia Screening & Triage Platform	Audit Date: 18 September 2026
Architecture: Vanilla ES6+ SPA/MPA + Node.js AI Proxy + Supabase DB	Compliance Scope: Malaysian PDPA 2010, COPPA & Educational Privacy

1. Executive Summary

LexiSense is a specialized, web-based early dyslexia pre-diagnosis and triage system for children aged 6–12. The platform establishes a unified ecosystem for **Parents, Educators/School SEN Coordinators, and Super Administrators**. By integrating subjective parent/teacher observations with objective biometric ocular-saccade tracking and real-time speech phoneme analysis, LexiSense calculates a calibrated 0–100% composite risk index, identifying reading differences before academic failure occurs.

2. Technology Stack & Component Inventory ("What We Use")

2.1 Client-Side Frontend Technologies

Category	Library / Technology	Exact Implementation & Role in LexiSense
Core Framework	HTML5 / Vanilla ES6+ JS	Modular architecture without compilation build-step. Handles state for Parent, Admin, Super Admin, and Wizard views.
UI Design Tokens	Tailwind CSS 3.x (Custom)	Accessible color schemes (Purple/Amber), responsive layouts, modal overlays, and visual feedback states.
Dyslexia Fonts	Lexend, OpenDyslexic, Baloo	Specialized letterform weighting to reduce visual crowding, character rotation, and letter-flipping effects.
Biometric Tracking	WebGazer.js 2.1	In-browser pupil tracking, ridge regression calibration, horizontal saccadic displacement detection (>35px).
Speech Recognition	Web Speech API	Oral reading speech-to-text decoding, phoneme error scoring, hesitation latency calculation, and WCPM tracking.
Data Visualization	Chart.js 4.x	Renders 5-domain cognitive radar charts, longitudinal screening histories, and class-wide risk distributions.
Document Engine	html2pdf.js & jsPDF	Client-side rendering and high-resolution export of clinical dossiers, IEP accommodation notes, and school triage reports.

2.2 Backend Gateway, AI & Cloud Infrastructure

Subsystem	Technology	Architecture & Integration Details
Node.js AI Proxy	Express 4.21 (`server.js`)	Handles `/api/chat` and `/api/send-email-report`. Role-conditioned system prompt injection for Ollie AI.
Conversational AI	OpenAI GPT-4o-mini	Powers Ollie the Wise Owl (🦉), delivering bilingual (English & Bahasa Melayu) dyslexia advice and triage guidance.
Email Dispatch	Nodemailer 10.0 (SMTP)	Automated dispatch of clinical dossiers to parents and educators with custom sender headers (`noreply@mykasih.com.my`).
Cloud Database	Supabase (PostgreSQL 15)	Primary relational data store (`profiles`, `children`, `students`, `screening_results`, `audit_logs`, `classes`, `schools`).
Extended Backend	Django 5.x REST Framework	Python backend alternative with `AbstractUser` modeling, raw coordinate time-series, and local SQLite persistence.

3. Scientific Methodology & Screening Pipeline ("How the Method Works")

LexiSense implements a **3-Pillar Multimodal Triangulation Engine**. By synthesizing three independent assessment channels, the system eliminates single-modality bias (such as test anxiety or webcam calibration noise).

Screening Pillar	Weight	Domain & Methodology	Quantitative Formula & Scaling
Pillar 1: Behavioral Survey	40%	30 Standardized Items across 5 clinical domains: Phonology, Decoding Fluency, Letter Reversals (b/d), Memory, and Family History. 4-point Likert scale (0=Never, 1=Mild, 2=Moderate, 3=Severe).	Excludes 'Not Observed / N/A' answers: Score = (Sum / (ValidItems * 3)) * 100
Pillar 2: Oral Fluency	40%	Standardized Reading Passage (English, Malay, Mandarin, Tamil). Web Speech API tracks reading duration, hesitations, and calculates Words Correct Per Minute (WCPM).	Benchmarked to Grade Norms (Z-Score): Z = (Expected - Observed) / SD Score = clamp(5, 98, 50 + (Z * 20))
Pillar 3: Ocular Saccades	20%	9-Point Calibrated WebGazer Tracking . Dual-stream EMA filtering (alpha=0.25). Detects line-aware saccadic regressions (>35px backwards, 280ms cooldown) and fixation dispersion.	Calculates regression deficit + dispersion: Score = min(50, RegRate*2.2) + max(0, 85-Stability)*0.8

Composite Risk Score Formula & Stratification Tiers:

Composite Risk Index = (Pillar 1 * 0.40) + (Pillar 2 * 0.40) + (Pillar 3 * 0.20)

**Note: If eye-tracking is not active or calibration quality is poor, the system falls back gracefully to validated questionnaire and oral fluency data.*

Composite Index	Risk Level Tier	Clinical Interpretation	Recommended Action Protocol
0% – 34%	Few Indicators	Reading decoding & fluency are age-appropriate.	Routine home reading & standard classroom curriculum.
35% – 64%	Some Indicators	Moderate phoneme hesitation or ocular tracking skips.	Multisensory phonics drills, reading focus ruler, teacher review.
65% – 100%	Elevated Indicators	Significant cross-pillar markers of dyslexia profile.	Formal diagnostic referral to an Educational Psychologist / SEN Lead.

3.2 User Portals & Operational Workflows

- **Parent Portal:** Child registration, home screening initiation, historical progress tracking, and downloadable PDF clinical dossiers.
- **Educator / Admin Portal:** Roster and cohort management, batch student screening triage, IEP classroom accommodation tracking, and teacher follow-up logs.
- **Super Admin Portal:** Multi-school governance, real-time audit logs, algorithm weight recalibration, user approval, and database synchronization.
- **Ollie the Wise Owl (AI):** Conversational support engine with context-aware prompt injection and local heuristic fallback when offline.

4. Comprehensive Security & Privacy Audit ("What the Security Is")

4.1 Current Security Controls in Place

- Role-Based Access Control (RBAC):** Hierarchical role separation between ``parent``, ``admin``, ``teacher``, and ``super_admin``.
- Route Authorization Guard (``auth-guard.js``):** Enforces session checks and role validation; masks page DOM (``visibility: hidden``) before rendering to prevent layout leakage.
- Client-Side Biometric Isolation:** WebGazer processes webcam video streams strictly in browser memory. No raw video footage or facial frames are uploaded to any server.
- Audit Trail Architecture:** Comprehensive ``audit_logs`` table tracking actors, events, targets, client IP addresses, and completion status.

4.2 In-Depth Vulnerability Assessment & Findings Matrix

Vulnerability	Severity	Technical Analysis & Code Reference	Required Remediation Action
Permissive Supabase RLS Policies	HIGH	In <code>supabase_schema.sql:223-250</code> , all RLS policies are set to <code>USING (true) WITH CHECK (true)</code> for both anon and authenticated. Any client with the anon key can query/modify all profiles, students, and results.	Restructure RLS policies to enforce ownership checks: <code>USING (auth.uid() = parent_id)</code> for parents, and role-verified JWT checks for school admins.
Public Anon Key Hardcoding	MEDIUM	Supabase URL and Anon Key are hardcoded in <code>frontend/js/supabase.js:7-8</code> . Without strict RLS, the entire cloud PostgreSQL database is open to automated scraping.	Lock down RLS policies immediately. Rotate Supabase publishable keys and load configurations via environment-driven deployments.
Unauthenticated AI & Email API	MEDIUM	In <code>backend/server.js:44, 117</code> , endpoints <code>/api/chat</code> and <code>/api/send-email-report</code> lack JWT verification and rate limiting. Vulnerable to OpenAI credit exhaustion and SMTP spam.	Install <code>express-rate-limit</code> (max 20 req/min). Add middleware verifying Supabase Auth Bearer tokens before executing AI completions or email dispatches.
Client-Side Auth Bypass Risk	LOW-MED	<code>auth-guard.js</code> permits <code>localStorage</code> -based demo roles (<code>`is_demo`</code>) and fallback user objects, enabling client-side DevTools tampering to render restricted UI views.	Ensure all database reads/writes and backend endpoints strictly validate server-side JWT tokens regardless of client UI state.

■■■ STATUTORY COMPLIANCE & PII PRIVACY MANDATE (PDPA 2010 / COPPA):

LexiSense stores sensitive pediatric data (names, ages, schools, audio readings, and dyslexia risk indices). Under the **Malaysian Personal Data Protection Act 2010** and international child protection principles: (1) Explicit, auditable parental consent must be logged prior to screening; (2) Data at rest must be encrypted via PostgreSQL AES-256; (3) Right-to-be-forgotten / profile deletion workflows must be strictly respected across cloud tables.

5. Hardening Implementation Plan & Engineering Roadmap

5.1 Proposed Production RLS SQL Configuration

To replace the permissive USING (true) policies, execute the following SQL hardening script in the Supabase SQL Editor:

```
-- 1. Restrict Profiles to User Ownership
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Profiles: Users can read/write own profile"
ON public.profiles FOR ALL TO authenticated
USING (auth.uid() = id) WITH CHECK (auth.uid() = id);

-- 2. Restrict Children to Registered Parents
ALTER TABLE public.children ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Children: Parents manage own children"
ON public.children FOR ALL TO authenticated
USING (auth.uid() = parent_id) WITH CHECK (auth.uid() = parent_id);

-- 3. Restrict Screening Results Access
ALTER TABLE public.screening_results ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Screenings: Parents view own results"
ON public.screening_results FOR SELECT TO authenticated
USING (auth.uid() = parent_id);
CREATE POLICY "Screenings: Authenticated users can insert results"
ON public.screening_results FOR INSERT TO authenticated
WITH CHECK (auth.uid() = parent_id);
```

5.2 Express Backend Middleware Hardening (`server.js`)

```
import rateLimit from 'express-rate-limit';

// IP Rate Limiter to prevent LLM credit draining & email flooding
const apiLimiter = rateLimit({
  windowMs: 1 * 60 * 1000, // 1 minute
  max: 25, // limit each IP to 25 requests per windowMs
  message: { error: 'Too many requests. Please try again in 60 seconds.' }
});
app.use('/api/', apiLimiter);
```

5.3 Actionable Milestone Roadmap

Phase	Milestone Target	Key Deliverables	Status / Priority
Phase 1	Security & RLS Hardening	Apply strict SQL RLS policies, rotate public API keys, add rate limiting to Express server.	Immediate (P0)
Phase 2	Backend Unification	Consolidate Django and Node services into a unified API gateway with centralized Supabase Auth.	Short-Term (P1)
Phase 3	Normative Expansion	Calibrate Grade 1–6 oral reading norms (WCPM) across multi-lingual national school datasets.	Medium-Term (P2)